So far I've used the terms *application servers* and *web servers*. Later, I will also mention *Servlet containers*. What's the difference?

- *Web server*—A server that handles HTTP requests coming from a client and replies with HTTP responses, such as Apache HTTPD.
- *Servlet container*—A component, part of a web server, that provides an execution context for web applications using the Java Servlet API (like Spring MVC apps). Tomcat (Catalina) is one example.
- *Application server*—A server that provides a complete execution environment (like Jakarta EE) for different types of applications and supports several protocols, such as WildFly.

3.2.1 Executable JARs and embedded servers

One of the differences between a traditional approach and a cloud native one is how you package and deploy applications. Traditionally, we used to have application servers or standalone web servers. They were expensive to set up and maintain in production, so they were used to deploy several applications, packaged as EAR or WAR artifacts for the sake of efficiency. Such a scenario created coupling among the applications. If any of them wanted to change something at the server level, the change would have to be coordinated with other teams and applied to all the applications, limiting agility and application evolution. Besides that, the deployment of an application depended on a server being available on the machine, limiting the application's portability across different environments.

When you go cloud native, things are different. Cloud native applications should be self-contained and not dependent on a server being available in the execution environment. Instead, the necessary server capabilities are included in the application itself. Spring Boot offers built-in server functionality that helps you remove the external dependency and make the